

2. Follow Naming Conventions:

- Use meaningful variable and function names.
- Follow a consistent naming convention, such as camelCase for variables and functions, and PascalCase for class names.

3. Manage Resources Properly:

- Use smart pointers (e.g., `std::unique_ptr`, `std::shared_ptr`) to manage dynamically allocated resources and prevent memory leaks.
- Follow the RAII (Resource Acquisition Is Initialization) principle to ensure resources are properly initialized and cleaned up.

4. Write Efficient Code:

- Optimize your code for performance by minimizing unnecessary computations and avoiding excessive memory allocations.
- Use algorithms and data structures from the STL to leverage well-optimized implementations.

5. Document Your Code:

- Write clear and concise comments to explain complex logic and assumptions.

- Use tools like Doxygen to generate API documentation from your code comments.

6. Test Thoroughly:

- Write unit tests to verify the correctness of your code.
- Use testing frameworks like Google Test to automate testing and ensure your code behaves as expected.

7. Use Version Control:

- Use version control systems like Git to manage your codebase.
- Commit changes regularly and use descriptive commit messages to track the evolution of your code.

In this chapter, you have learned how to set up your C++ development environment, explored basic syntax, and written a simple C++ program. Additionally, we discussed advanced features of C++ and industry trends, ensuring you are well-equipped to build efficient and high-performance applications.

As you continue your coding journey, C++ will be an invaluable tool in your development toolkit. The knowledge and skills you have gained in this chapter will serve as a solid foundation for exploring more advanced topics and applications in C++. 

C++ STANDARD LIBRARY COMPONENTS CHEAT SHEET

Here's a cheat sheet that summarizes essential components of the C++ Standard Library. This table includes descriptions and syntax examples to help you quickly reference and utilize C++ library components in your projects.		
Component	Description	Syntax Example
Vector	A dynamic array that can grow in size.	<code>std::vector<int> vec = {1, 2, 3};</code>
List	A doubly linked list.	<code>std::list<int> lst = {1, 2, 3};</code>
Map	An associative container that stores key-value pairs.	<code>std::map<std::string, int> map = {{"one", 1}, {"two", 2}};</code>
Set	A collection of unique elements.	<code>std::set<int> mySet = {1, 2, 3};</code>
Stack	A container adapter that provides stack functionality.	<code>std::stack<int> stack; stack.push(1);</code>
Queue	A container adapter that provides queue functionality.	<code>std::queue<int> queue; queue.push(1);</code>
Priority Queue	A container adapter that provides priority queue functionality.	<code>std::priority_queue<int> pq; pq.push(1);</code>
Algorithm	Functions for performing various operations on containers.	<code>std::sort(vec.begin(), vec.end());</code>
Iterator	An object that points to an element in a container.	<code>std::vector<int>::iterator it = vec.begin();</code>
String	A sequence of characters.	<code>std::string str = "Hello, World!";</code>
Utility	Commonly used utility functions and classes.	<code>std::pair<int, int> p = std::make_pair(1, 2);</code>
Chrono	A library for dealing with time.	<code>std::chrono::system_clock::now();</code>
Thread	Support for multithreading.	<code>std::thread t([]{ std::cout << "Hello from thread!"; });</code>